

IHK – InsertionSort: Fragen & Antworten

Prüfungsjahre (sichtbar in ähnlicher Form): 2019, 2020/21 (Wi), 2022 (So), 2023 (Wi/So), 2024

Pseudocode-Regeln (IHK-Stil): **Funktion ... : Typ**, Zuweisung =, Schleifen: **für i = ... bis ...**, **solange, wenn ... dann, sonst, gib ... zurück**.

I1 – Standard InsertionSort (aufsteigend, Ganzzahlen)

Gekommen: Sommer 2019, Winter 2020/21

Aufgabe (IHK-Stil): Sortieren Sie `werte: Array<Integer>` aufsteigend mittels Einfügen. Geben Sie Pseudocode an.

Lösung (Pseudocode)

```
Funktion insertionSort(werte: Array<Integer>) : void
    für i = 1 bis länge(werte) - 1
        key = werte[i]
        j = i - 1
        solange j >= 0 und werte[j] > key
            werte[j + 1] = werte[j]
            j = j - 1
        ende solange
        werte[j + 1] = key
    ende für
Ende Funktion
```

Antwort (Java-ähnlich)

```
void insertionSort(int[] a) {  
    for (int i = 1; i < a.length; i++) {  
        int key = a[i];  
        int j = i - 1;  
        while (j >= 0 && a[j] > key) {  
            a[j + 1] = a[j];  
            j--;  
        }  
        a[j + 1] = key;  
    }  
}
```

I2 – Absteigend sortieren

Gekommen: Winter 2023

Aufgabe: Passen Sie den Algorithmus an, sodass **absteigend** sortiert wird.

Pseudocode

```
Funktion insertionSortAbsteigend(werte: Array<Integer>) : void
    für i = 1 bis länge(werte) - 1
        key = werte[i]
        j = i - 1
        solange j >= 0 und werte[j] < key
            werte[j + 1] = werte[j]
            j = j - 1
        ende solange
        werte[j + 1] = key
    ende für
Ende Funktion
```

Java-ähnlich

```
void insertionSortDesc(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] < key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

I3 – Objekte/Strings sortieren (Attribut / Comparator)

Gekommen: Sommer 2022 (Objekt-Attribut), Winter 2023 (Namen)

Aufgabe A: Sortieren Sie `namen: Array<String>` alphabetisch (aufsteigend).

Pseudocode (Strings)

```
Funktion insertionSortStrings(namen: Array<String>) : void
    für i = 1 bis länge(namen) - 1
        key = namen[i]
        j = i - 1
        solange j >= 0 und namen[j] > key      // lexikographisch
            namen[j + 1] = namen[j]
            j = j - 1
        ende solange
        namen[j + 1] = key
    ende für
Ende Funktion
```

Aufgabe B: Sortieren Sie `kurse: Array<Tageskurs>` nach `preis` (aufsteigend).

Pseudocode (Objekt-Attribut)

```
Funktion insertionSortByPreis(kurse: Array<Tageskurs>) : void
    für i = 1 bis länge(kurse) - 1
        key = kurse[i]
        j = i - 1
        solange j >= 0 und kurse[j].preis > key.preis
            kurse[j + 1] = kurse[j]
            j = j - 1
        ende solange
        kurse[j + 1] = key
    ende für
Ende Funktion
```

Java-ähnlich mit Comparator

```
void insertionSort(Tageskurs[] k, java.util.Comparator cmp) {  
    for (int i = 1; i < k.length; i++) {  
        Tageskurs key = k[i];  
        int j = i - 1;  
        while (j >= 0 && cmp.compare(k[j], key) > 0) {  
            k[j + 1] = k[j];  
            j--;  
        }  
        k[j + 1] = key;  
    }  
}
```

I4 – Simulation (zwei äußere Runden)

Gekommen: Winter 2020/21, Sommer 2024

Aufgabe: Simulieren Sie InsertionSort für [8, 3, 5, 2] nach den ersten zwei äußeren Runden (i=1 und i=2).

Lösungsschritte

i	key	Verschiebungen	Array danach
1	3	8 → nach rechts	[3, 8, 5, 2]
2	5	8 → nach rechts	[3, 5, 8, 2]

Nächster Schritt (i=3): key=2; 8→rechts, 5→rechts, 3→rechts → [2, 3, 5, 8] .

I5 – Fehleranalyse (typische IHK-Fallen)

Gekommen: Sommer 2022 (Korrektur Einfügelogik), Winter 2023 (Index)

F1 – Einfügeindex falsch (j/j+1 vertauscht)

FALSCH

```
solange j >= 0 und werte[j] > key
    werte[j] = werte[j + 1]    // überschreibt nach links
    j = j - 1
ende solange
werte[j] = key                // falsche Position
```

RICHTIG

```
solange j >= 0 und werte[j] > key
    werte[j + 1] = werte[j]
    j = j - 1
ende solange
werte[j + 1] = key
```

F2 – Startindex der äußeren Schleife

FALSCH: für i = 0 bis länge(werte) - 1

RICHTIG: für i = 1 bis länge(werte) - 1

F3 – Absteigend vergessen (Vergleichszeichen)

aufsteigend: solange j >= 0 und werte[j] > key

absteigend: solange j >= 0 und werte[j] < key

F4 – Strings/Objekte mit „==“ sortiert

Strings: solange j >= 0 und namen[j] > key

Java: while (j >= 0 && namen[j].compareTo(key) > 0) { ... }

I6 – Theorie-Check (kurz & knackig)

1. **Laufzeit:** Best $O(n)$, Average/Worst $O(n^2)$
2. **Stabil?** Ja – gleiche Schlüssel behalten Reihenfolge
3. **Speicher?** $O(1)$ zusätzlich (in-place)
4. **Vorteile:** sehr gut für kleine / fast sortierte Daten; einfache Implementierung
5. **Nachteile:** langsam bei großen unsortierten Datenmengen

Merksatz: „Nimm das nächste Element (`key`), schiebe größere links nach rechts, setze `key` an `j+1` ein.“